Introduction to Database Systems
Spring 2025

Lecture 9

Omar Shahbaz Khan

General Info

- Homework 3 deadline today 23:59
- Homework 4 will be out Thursday or Friday
  - Announcement will be made on Piazza when it is up
  - Deadline will be two weeks from its release date

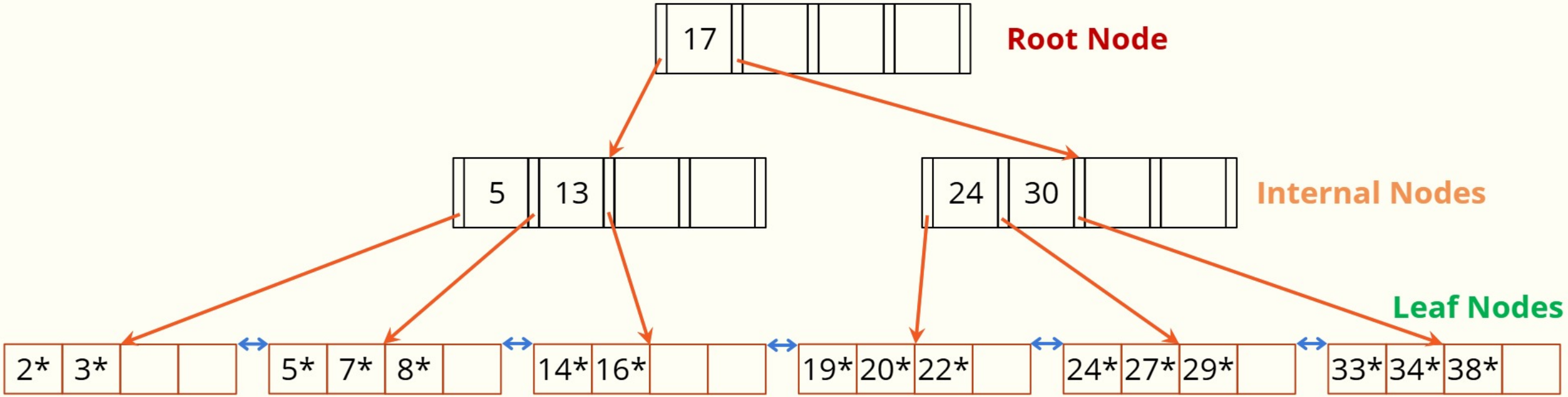
TODO

- B+ Tree Index
  - Search
  - Insert
  - Storage footprint
- JSON in Databases
  - Specialized Databases or RDBMS
  - JSON in DuckDB
- Transactions
  - ACID

B+ Tree Index

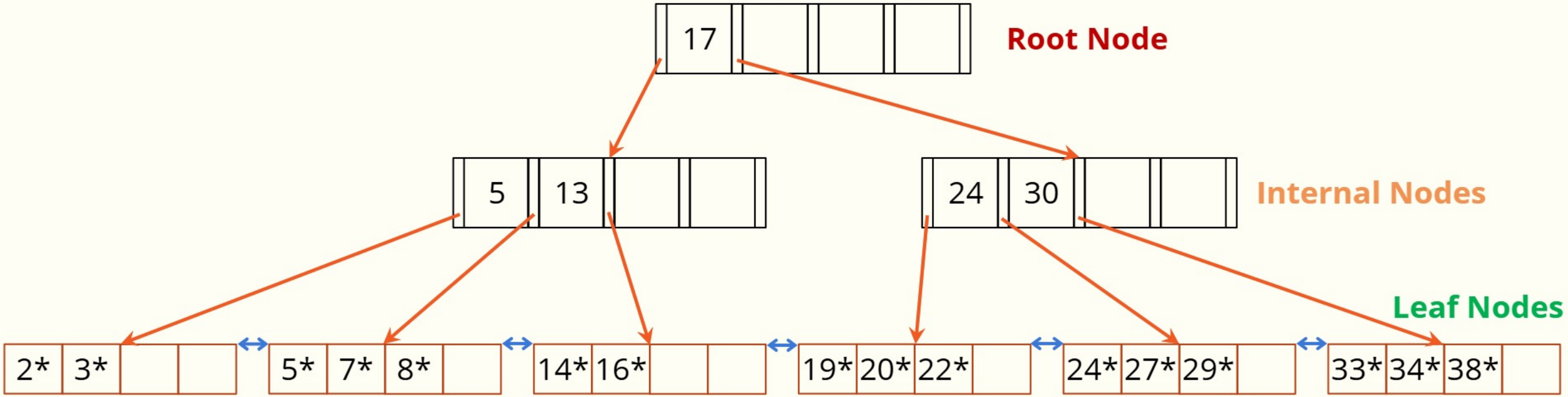
- A balanced tree data structure
  - Do not get it confused with binary trees
- The most common index type
  - … in relational databases
- Supports both point and range queries
  - Recall Hash Indexes only supports point queries
- Dynamic structure
  - Adapt to insertions and deletions
  - Maintains a balanced tree

A Sample B+ Tree Index



- Nodes/Leafs have a maximum and minimum capacity (order)
  - Can refer to number of children or keys, depending on textbooks and online articles
  - Typically it is the number of children, so in this case we have an order = 5
- Leafs are connected through a linked list
- Each node is one disk page

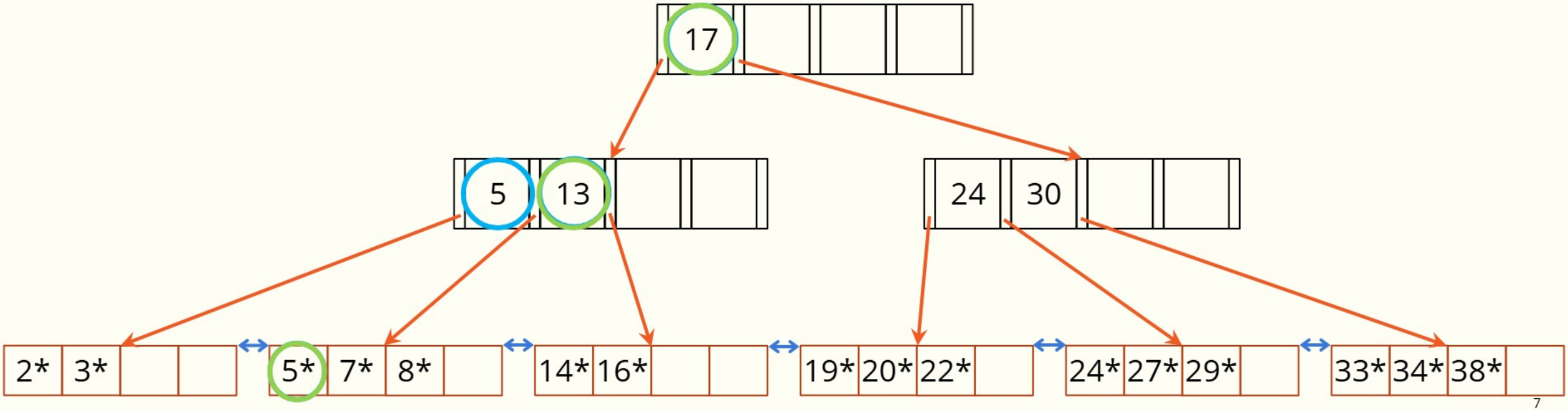
A Sample B+ Tree Index



- X* represents (search key, pointer list) pair e.g. (X, [address of tuple X_1,...])
- Why does it have a pointer list and not a single pointer?
  - To support trees built around non-unique keys

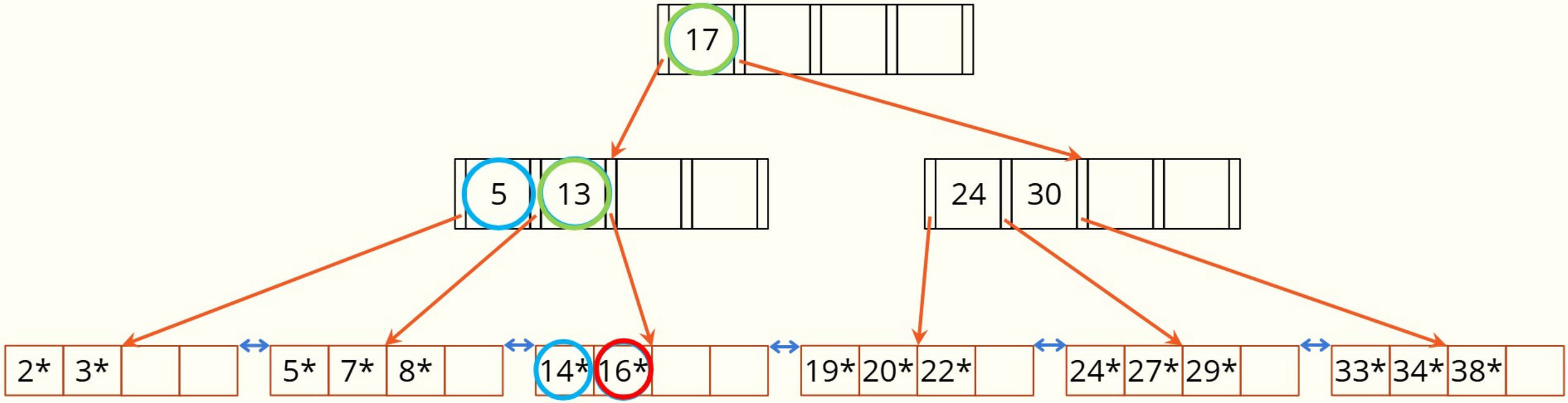
Searching

- Compare search key (SK) against node key element (K)
  - If SK < K Go to the left child node of K
  - Else check K+1
  - If the final element is reached and is still not passing the statement, go to the right-most child node
- Example: Find 5*



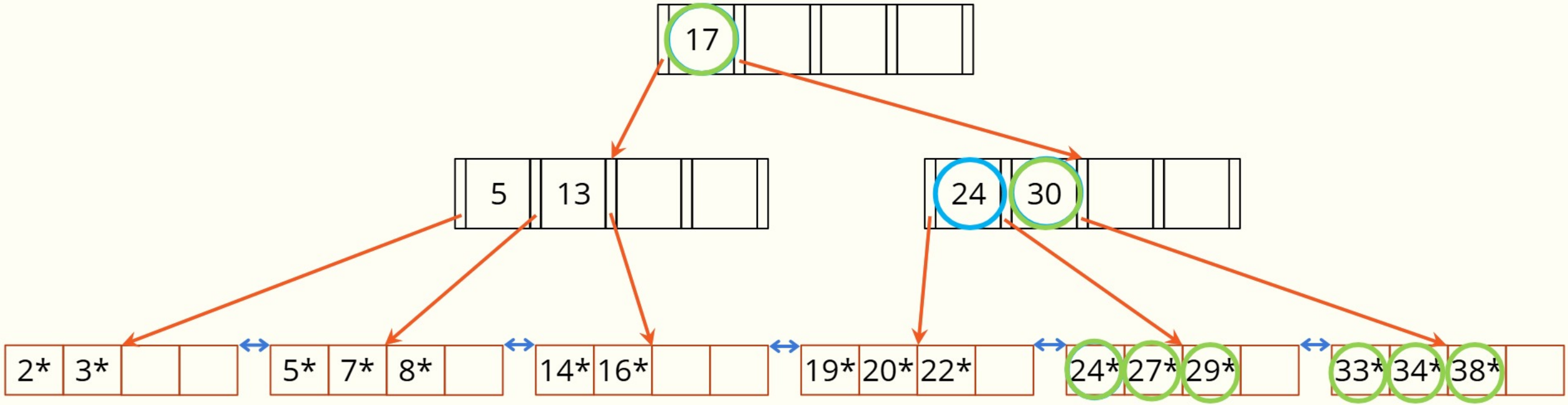
Searching

- Example: Find 15*



Searching

- Example: Find >= 24*

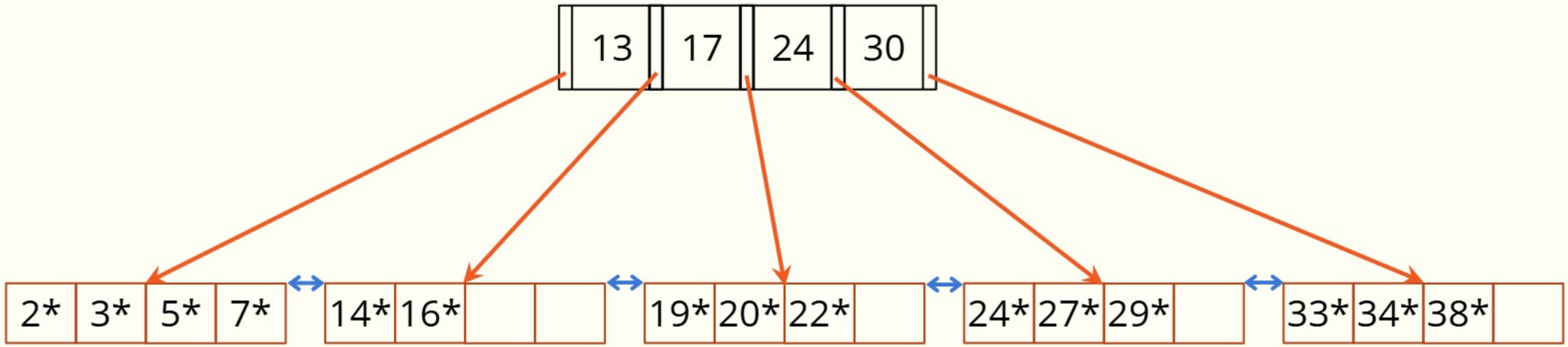


Intra-Node Searching

- In the examples we have been using scan
- B+ Tree nodes have hundreds/thousands of key values in the node
- Use binary search!

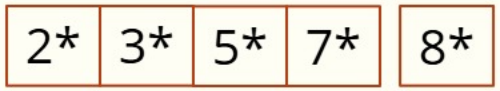
Inserting a new element

- Consider the following B+ Tree
- We want to insert 8*
- Follow search strategy to find appropriate leaf node
  - Leaf node is full, must therefore split
  - Root node is full, must therefore split

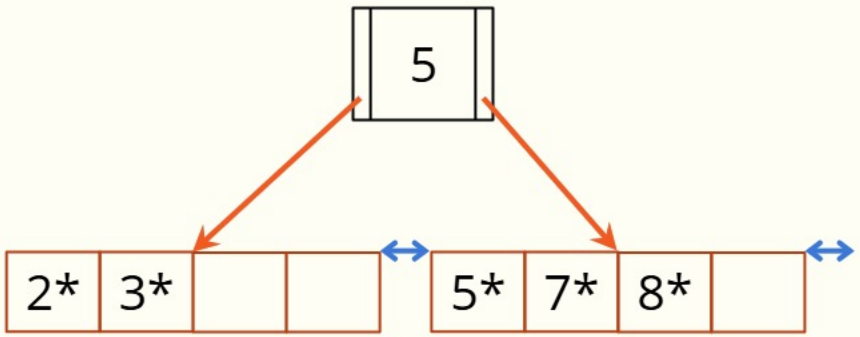


Inserting 8*: Split the leaf node

- When splitting a node you have to assume that the new element has been inserted
  - Sort the elements



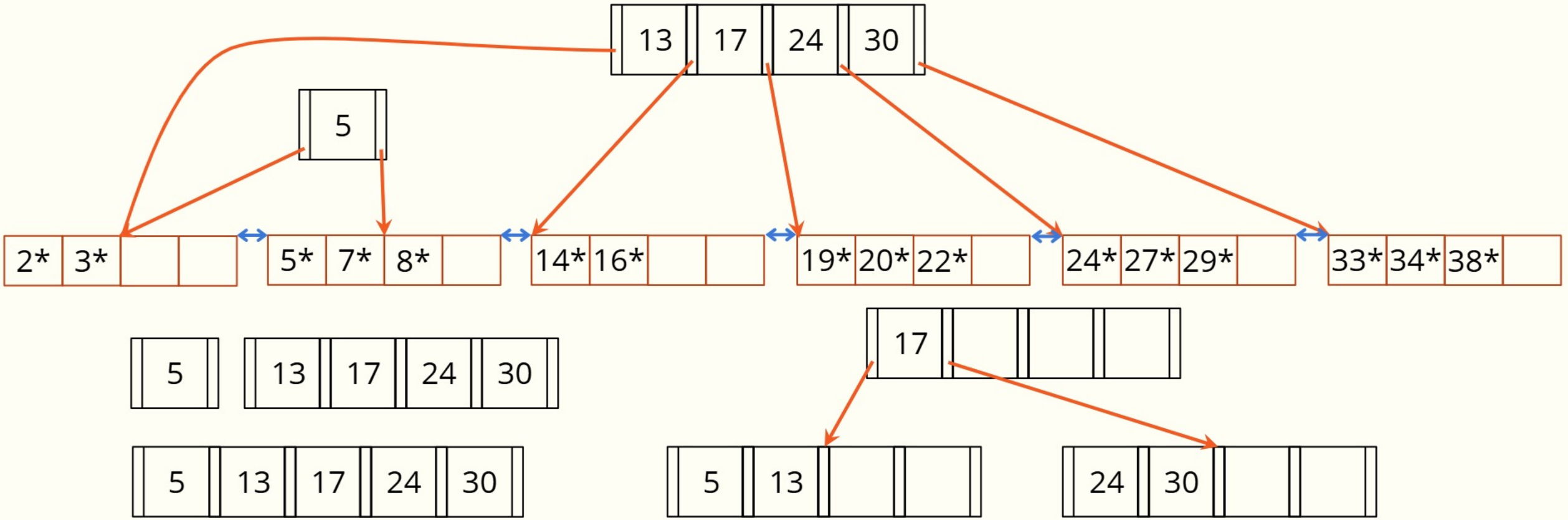
- The middle element becomes the parent of the new nodes (5)



- The linked list is updated

Inserting 8*: Intermediate state

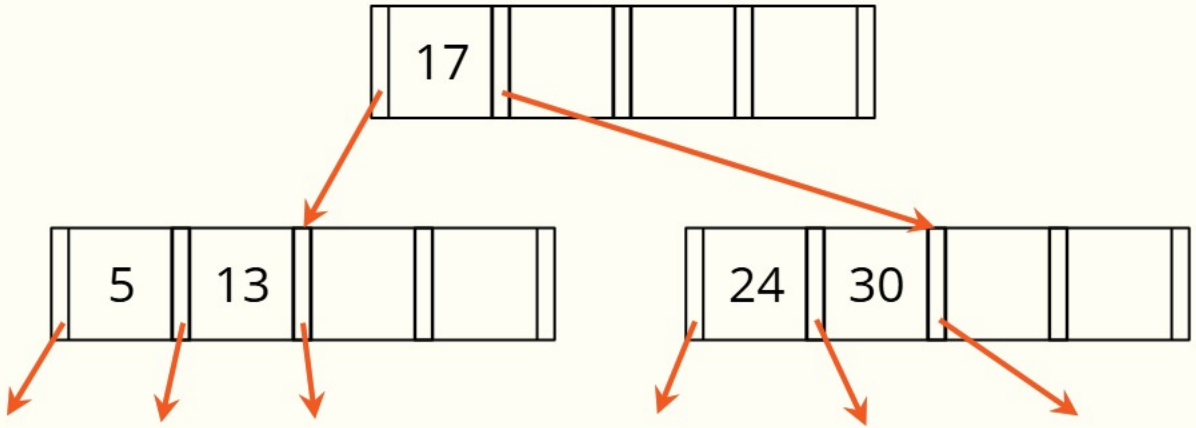
- No room for 5 in the root node :(



Inserting 8*: Split the root node

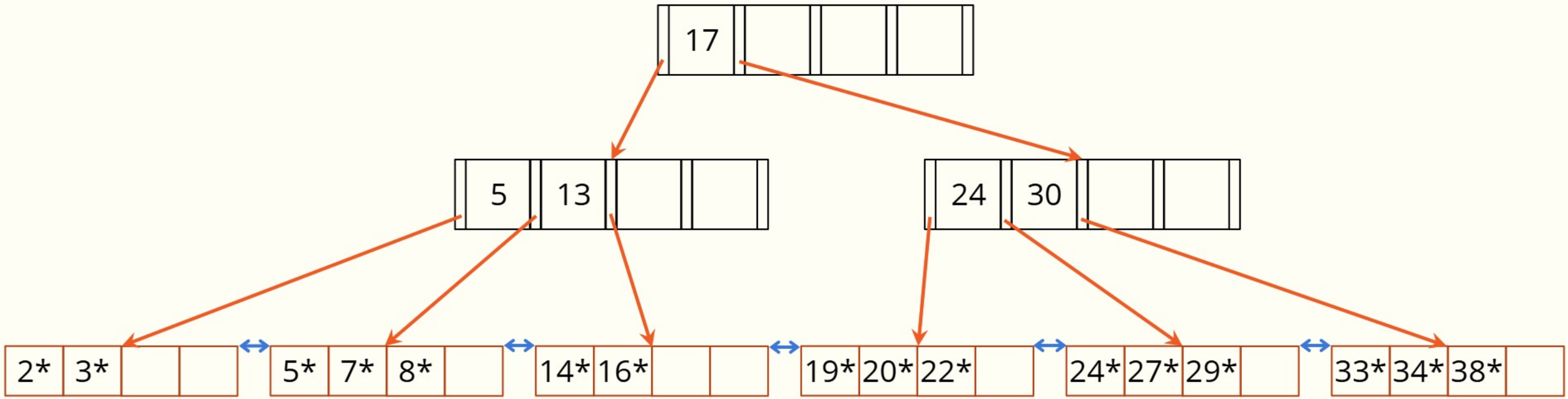
- When splitting a node you have to assume that the new value has been inserted
  - Sort the elements

- The parent of these new nodes will be 17



After Inserting 8*

- Tree grows wider, then higher



Storage

- A typical tree
  - Order: 1000 (~=16 KB per page / 16 bytes per entry)
  - Utilization is about 67% in practice
  - Fanout: 670
- Capacity
  - Root = 670 records
  - Two levels = 670^2 = 448,900 records
  - Three levels = 670^3 = 300,763,000 records
  - Four levels = 670^4 = 201,511,210,000 records
- Top levels may fit in memory
  - Level 1 = 1 page = 16 KB
  - Level 2 = 670 pages = 11 MB
  - Level 3 = 448,900 pages = 7 GB

JSON

What is JSON?

- From DuckDB:

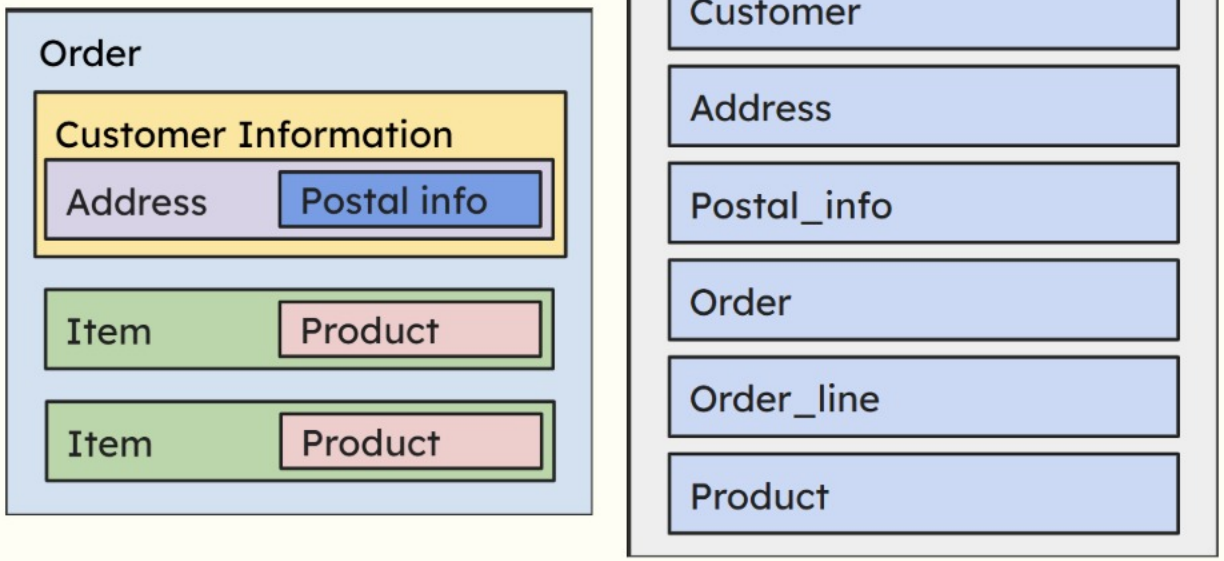
"JSON is an open standard file format and data interchange format that uses **human-readable** text to store and transmit data objects consisting of **attribute–value pairs and arrays** (or other **serializable values**). While it is *not a very efficient format for tabular data*, it is very commonly used, especially as a data interchange format."

JSON (JavaScript Object Notation)

- A semi-structured data structure
- It is a human readable and easy to interpret format
- Commonly used for transmitting data between a server and web applications.
- Enables grouping of related data into a single, organized "document".
- JSON databases / Document Databases
  - NoSQL database
  - Stores JSON documents instead of rows in tables

JSON Object

```
{
   "order_id": 3294,
   "customer": {
      "ssn": "123456789",
      "name": "John Doe",
      "address": {
               "name": "123 Random Street",
               "postal_code": "98765"
      }
   },
   "line-items": [
      {"product": "Sofa", "price": 2500},
      {"product": "Table", "price": 1000},
   ]
}
```



JSON object requires one select on the order_id to get all information, while the relational way requires joining several tables.

Document Databases

- Flexibility (schemaless) vs Rigid (enforced schemas)
  - Understand your business requirements
  - Being schemaless can be dangerous if not kept in check
- Types of NoSQL databases:
  - Document Databases (MongoDB, Couchbase, Amazon DocumentDB, FerretDB)
  - Graph Databases (Neo4j, InfiniGraph, Allegro)
  - Key-Value Databases (Redis, Apache Cassandra)
- NoSQL databases excel in specialized cases
  - Handling large volumes of unstructured data
  - Horizontal scaling
  - Real-time analytics
- Around 10–15 years ago, NoSQL databases were all the hype.
  - Some developers were even proclaiming we didn't have to learn RDBMS or SQL anymore
  - So did NoSQL kill relational databases?
  - It’s 2025 now, and RDBMSs are still a core part of database courses — and widely used in industry

Relational Databases added support for JSON

- Modern relational databases bridge the gap to NoSQL
  - Added JSON support
  - Horizontal scaling
  - Flexible schemas
  - Real-time analytics
  - PostgreSQL, MySQL, DuckDB, etc. can all read and query JSON
- JSON data type
  - Store JSON directly into a column
  - Great for web services where data is exchanged using JSON objects
  - Previously you would have to extract each value from the object and use an insert statement
- PostgreSQL has been shown to outperform MongoDB (most popular JSON DB)
- FerretDB
  - A document database that is written on top of PostgreSQL 🤔
  - Uses Microsoft's DocumentDB PostgreSQL open-source extension
  - Open-source alternative to MongoDB

JSON in DuckDB

- Reading JSON files
- JSON Functions
- Writing data to JSON files

Reading a JSON file

- Simple SELECT statement
  - SELECT * FROM 'my_file.json';
- For better transparency and additional features use the function read_json()
  - SELECT * FROM read_json('my_file.json')
- Specify which columns (keys) to get
  - SELECT * FROM read_json('my_file.json', **columns=**{key_1: 'DataType', key_2: 'DataType'});
- Specify the format of the file using 'newline_delimited' / 'array' / 'unstructured'
  - 'newline_delimited' = A JSON object per new line in file (alternatively, use read_ndjson())
  - 'array' = The file contains a JSON array
  - 'unstructured' = Reads valid JSON objects that are not in the other formats
  - For more information on formats see: https://duckdb.org/docs/stable/data/json/format_settings

Reading JSON into table

- CREATE TABLE json_test AS SELECT * FROM read_json('examples.json');
  - Note that the table will infer all column types
  - For more control over this we can use the **columns** to specify exact types
  - There will be no key or unique columns in the table
  - We can use ALTER TABLE to modify after the fact
- Alternative if columns and characteristics are known
  - Create table first with all the correct specifications
  - Insert rows using read_json()
  - CREATE TABLE json_test (id INT PRIMARY KEY, name VARCHAR, items VARCHAR[]);
  - INSERT INTO json_test SELECT * FROM read_json('characters.json');

DEMO 1

Extracting JSON values from objects

- json_extract() or use '->>'
- ```
  SELECT name, archenemy
  FROM read_json('examples_array.json')
  WHERE name = 'Harry Potter';
  ```

name varchar	archenemy struct("name" varchar, weakness varchar)
Harry Potter	{'name': Voldemort, 'weakness': Horcruxes}

- ```
  SELECT name, archenemy->>'$.name', archenemy->>'$.weakness'
  FROM read_json('examples_array.json')
  WHERE name = 'Harry Potter';
  ```

name varchar	(archenemy ->> '$.name') varchar	(archenemy ->> '$.weakness') varchar
Harry Potter	Voldemort	Horcruxes

Extracting JSON array elements

- ```
  SELECT name, friends
  FROM read_json('examples_array.json')
  WHERE name LIKE '%Skywalker';
  ```

name varchar	friends varchar[]
Anakin Skywalker	[Obi-Wan Kenobi, Ahsoka Tano, Yoda, Senator Palpatine]

- ```
  SELECT name, friends->>'$[2]'
  FROM read_json('examples_array.json')
  WHERE name LIKE '%Skywalker';
  ```

name varchar	(friends ->> '$[2]') varchar
Anakin Skywalker	Yoda

- Could also use friends[2], but then it would be treated as 1-indexed, whereas friends->>'$[2]' treats the list as 0-indexed.
- If the array contains objects, use ->>'$[index].key' to extract specific keys from an array of objects

Additional Functions

- json_array_length()
  - `SELECT name, json_array_length(weapons) AS number_of_weapons FROM read_json('examples_array.json');`
- json_keys()
  - Gets the keys from a JSON type
  - `SELECT json_keys(archenemy) FROM read_json('examples_array.json');`
- json_group_array() and json_group_object()
  - Convert a table column into an array
  - Convert two table columns into a single key-value object

Writing JSON

- Writing entire tables
  - `COPY <table_name> TO 'my_file.json';`
- Writing a query result into a file
  - `COPY (<query>) TO 'my_file.json';`
- To write a JSON array of objects use (ARRAY)
  - `COPY (<query>) TO 'my_file.json' (ARRAY);`

DEMO 2

Transactions

Consider the following transactions on the relation accounts (nr, balance, type):

Transaction A

```
UPDATE accounts SET balance=balance*1.02 WHERE type='savings';
UPDATE accounts SET balance=balance*1.01 WHERE type='salary' AND balance > 0;
UPDATE accounts SET balance=balance*1.07 WHERE type='salary' AND balance < 0;
```

Transaction B

```
UPDATE accounts SET type='salary' WHERE nr=12345;
```

Transaction C

```
UPDATE accounts SET balance=balance-1000 WHERE nr=12345;
```

Assume that account 12345 starts as 'savings' with balance=500.
What are the possible balance values after running transactions A, B and C?

A→B→C: **A→C→B:** **B→A→C:**

B→C→A: **C→A→B:** **C→B→A:**

Consider the following transactions on the relation accounts (nr, balance, type):

Transaction A

```
UPDATE accounts SET balance=balance*1.02 WHERE type='savings';
UPDATE accounts SET balance=balance*1.01 WHERE type='salary' AND balance > 0;
UPDATE accounts SET balance=balance*1.07 WHERE type='salary' AND balance < 0;
```

Transaction B

```
UPDATE accounts SET type='salary' WHERE nr=12345;
```

Transaction C

```
UPDATE accounts SET balance=balance-1000 WHERE nr=12345;
```

Assume that account 12345 starts as 'savings' with balance=500.
What are the possible balance values after running transactions A, B and C?

A→B→C: -490 **A→C→B:** -490 **B→A→C:** -495

B→C→A: -535 **C→A→B:** -510 **C→B→A:** -535

Consider the following transactions on the relation accounts (nr, balance, type):

Transaction A

```
UPDATE accounts SET balance=balance*1.02 WHERE type='savings';
UPDATE accounts SET balance=balance*1.01 WHERE type='salary' AND balance > 0;
UPDATE accounts SET balance=balance*1.07 WHERE type='salary' AND balance < 0;
```

Transaction B

```
UPDATE accounts SET type='salary' WHERE nr=12345;
```

Transaction C

```
UPDATE accounts SET balance=balance-1000 WHERE nr=12345;
```

Assume that account 12345 starts as 'savings' with balance=500 and they are interleaving.

A.1: 510
B: 510 (salary)
A.2: 515.10
C: -484.90
A.3: -518.843

Transactions

- Transactions group a set of database operations
  - Atomic, all operations succeed or none do
- May require shared database resources (rows, columns, tables)
  - Can lead to conflicts with multiple concurrent transactions
- Can be run sequentially or concurrently
  - Trade-off between consistency (data integrity) and performance (latency)

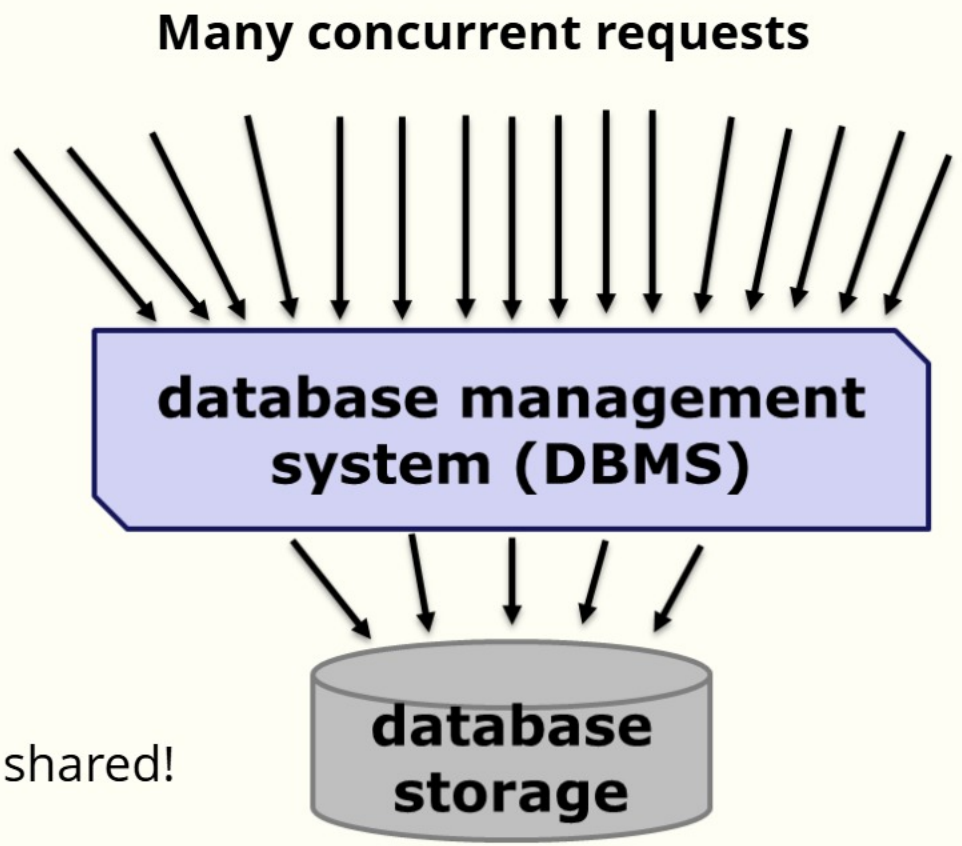
Transactions in SQL

- Use a BEGIN; COMMIT/ROLLBACK; block to group SQL statements into a transaction

```
BEGIN;
<SQL Statement 1>;
<SQL Statement 2>;
COMMIT; or ROLLBACK;
```

- COMMIT if the transaction runs as expected
- ROLLBACK if you encounter an error
  - This can also be used for testing
  - Make changes to the database tables, check if logic is correct, rollback to discard changes
- Save points are also available in some databases
  - Allows you to rollback state to a specific point in your running transaction
  - SAVEPOINT <sp_name>; ROLLBACK TO <name>;

Why do we need Transactions?

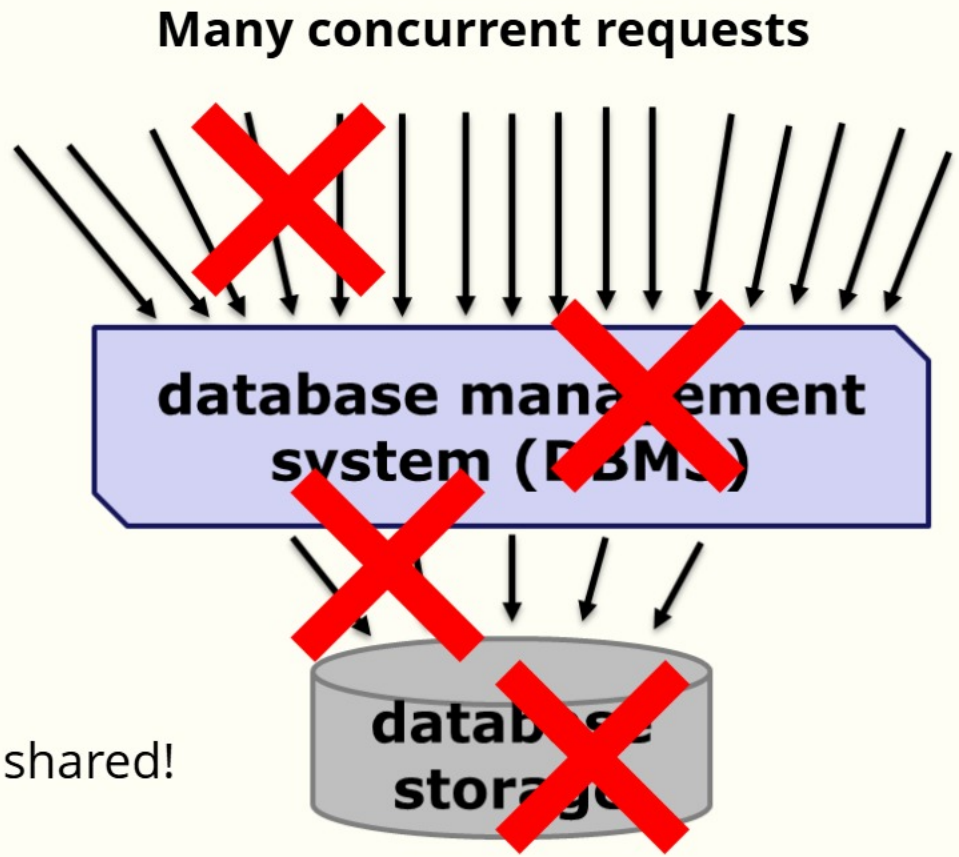


Data is shared!

DBMS must ensure reliable operations over shared data despite many concurrent accesses

Why do we need Transactions?

ANYTHING CAN CRASH!



Data is shared!

DBMS must ensure reliable operations over shared data despite many concurrent accesses

To the end user everything is fine

ATOMICITY

A Transaction is "one operation"

Each transaction runs to completion or has no effect at all

CONSISTENCY

Each Transaction moves the DB from one consistent state to another

After a transaction completes, the integrity constraints are satisfied

ISOLATION

Each Transaction is alone in the world

Transactions executed in parallel have the same effect as if they were executed sequentially

DURABILITY

Persistence of successful transactions even through system failure

The effect of a committed transaction remains in the database even if the system crashes

How to implement Transactions?

- Consistency ~= satisfying constraints
  - Use indexes for primary and foreign key, check statements, …
- Atomicity and Durability = tracking changes
  - Logging “before” values to UNDO changes
  - Logging “after” values to REDO changes
- Isolation = preventing corrupting changes
  - Pessimistic: Locking to prevent conflicts
  - Optimistic: Time stamps to detect conflicts
- More details in the next lecture :)

Takeaways!

- B+ Tree Indexes are predominantly used in OLTP databases
  - Has pointers to actual data at the leaf level
  - It's balanced structure makes it easy to determine storage requirements
  - Great for handling both range and point queries
  - Clustered vs Unclustered
- JSON is a semi-structured data format
  - Human-readable
  - Used for transmitting data between a server and web applications
  - Groups information into a document / object
  - Good support to handle JSON in RDBMS
- Transactions
  - ACID properties ensure database operations are run reliably and consistently